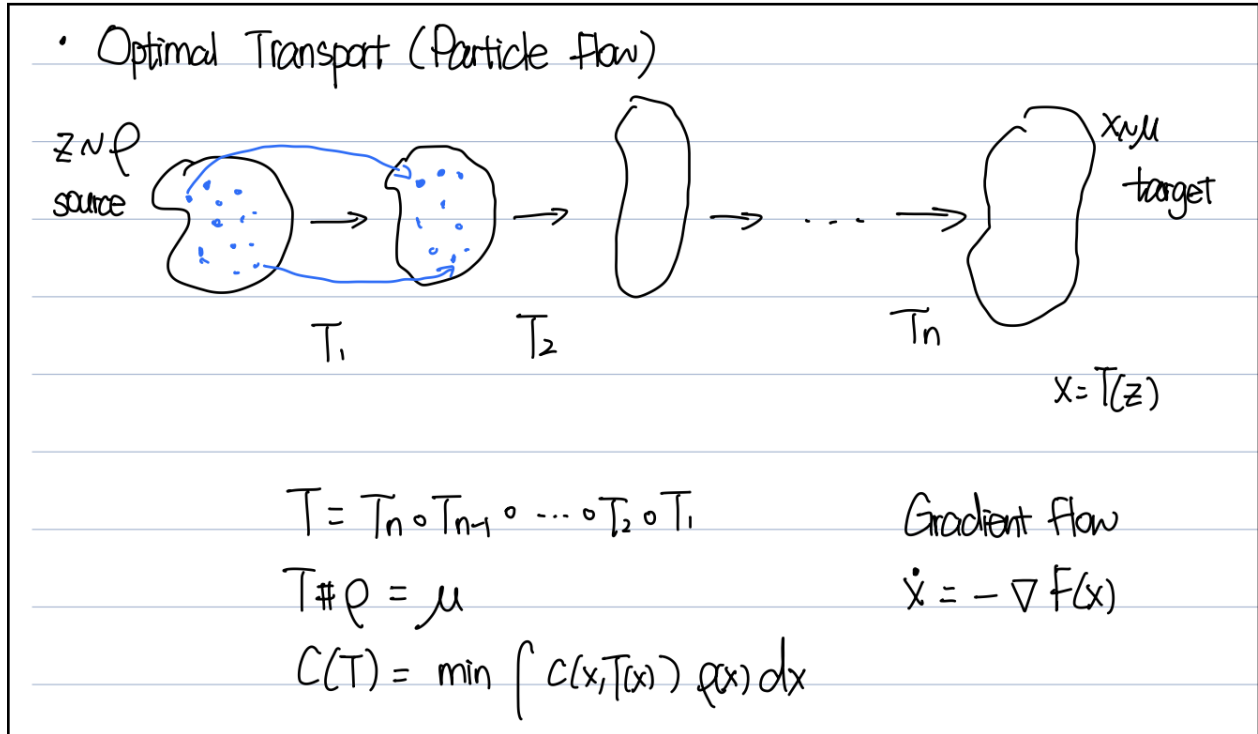


Tanya

Polymath Jr

07/19/2025



Discrete Transport Maps and Gradient-Based Particle Flow

We begin with the discrete map composition:

$$T = T_n \circ \dots \circ T_1$$

and the gradient-based particle update:

$$z^{(t+\Delta t)} = z^{(t)} + \Delta t \left(\nabla_z \log \mu(z^{(t)}) - \nabla_z \log \rho_t(z^{(t)}) \right).$$

If both are intended to describe the same underlying flow, then their connection must be made explicit.

The Connection

Each small transport map T_i corresponds to a gradient-based transformation of the form:

$$T_i(z) = z + \Delta t \cdot (\nabla_z \log \mu(z) - \nabla_z \log \rho_t(z)).$$

This directly mirrors the boxed update rule provided previously.

Composition of Steps

Applying these maps sequentially yields the overall transformation:

$$T = T_n \circ T_{n-1} \circ \dots \circ T_1.$$

This composition approximates the solution to the continuous-time gradient flow ordinary differential equation

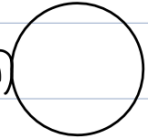
$$\frac{dz}{dt} = \nabla_z \log \mu(z) - \nabla_z \log \rho(z),$$

which solves for $z(t)$.

• Normalizing Flow (Parametrized Map)

source
 μ

(Gaussian)



$\rightarrow$

$T_1(\cdot; \theta_1)$



$\rightarrow$

$T_2(\cdot; \theta_2)$

$\dots$

$\rightarrow$

$T_n(\cdot; \theta_n)$



target
 ρ

$T = T_n \circ T_{n-1} \circ \dots \circ T_2 \circ T_1$
 $T(\cdot; \theta_1, \theta_2, \dots, \theta_n)$
 $\underbrace{\hspace{1.5cm}}_{\theta}$

Gradient flow

 $\dot{\theta} = -\nabla L(\theta)$

satisfy $T\#\mu = \rho$

$T_1, \dots, T_n$ invertible, differentiable

Key Difference in Definition

	Normalizing Flow	Particle Flow
Mechanism	Parameterized, invertible maps trained by gradient descent	Continuous (or discretized) particle movement, often gradient-based
Form	$T(z; \theta)$, composed of neural/analytic transforms	ODE/PDE-driven flow (e.g., Langevin, gradient flow)
Training Goal	Match transformed density to data distribution	Evolve particles so their empirical distribution matches the target
Invertibility	Required (to compute exact density via change of variables formula)	Not always required

• Gradient Descent: $\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla L(\theta) \big|_{\theta^{(k)}}$

• How to find a loss Objective L ?

If $T\#\mu = \rho$, let $\begin{cases} x = T(z) \sim \rho \\ z = T^{-1}(x) \sim \mu \end{cases}$

$$p(T(z)) |J_T(z)| = \mu(z)$$

$$\Rightarrow p(x) |J_{T^{-1}}(T^{-1}(x))|^{-1} = \mu(T^{-1}(x))$$

$$\Rightarrow p(x) = \mu(T^{-1}(x)) |J_{T^{-1}}(T^{-1}(x))|^{-1}, \text{ where } T \text{ and } T^{-1} \text{ dep. on } \theta$$

At obs time t , $T(\cdot, t)\#\mu = T_t(\cdot)\#\mu = \rho_t$

want ρ_t close to target ρ w/ samples $\{x_i\}_{i=1}^N \sim \rho$

$\Rightarrow \max \log \text{likelihood} \Leftrightarrow \min \text{negative log likelihood}$

$$\min_{\theta} L(\theta) = -\sum_{i=1}^N \log \rho_t(x_i; \theta)$$

$$= -\sum_{i=1}^N \log(\mu(T_t^{-1}(x_i; \theta)) |J_{T_t^{-1}}(T_t^{-1}(x_i; \theta))|)$$

Gradient Descent Update Rule

The gradient descent update rule for minimizing a loss function $\mathcal{L}(\theta)$ is given by:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla \mathcal{L}(\theta) \big|_{\theta^{(k)}}$$

Explanation of Terms

- $\theta^{(k)}$: The current value of the parameter vector at iteration k . This might represent model weights, transformation parameters, or other learnable variables.
- $\theta^{(k+1)}$: The updated parameter vector for the next iteration.
- η : The learning rate (also called the step size). This scalar controls how far to move along the gradient direction. A value that is too large may cause divergence, while a value that is too small may result in

slow convergence.

- $\nabla \mathcal{L}(\theta)|_{\theta^{(k)}}$: The gradient of the loss function evaluated at $\theta^{(k)}$. This is a vector of partial derivatives:

$$\left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \dots, \frac{\partial \mathcal{L}}{\partial \theta_d} \right) \Big|_{\theta=\theta^{(k)}}$$

It points in the direction of steepest *increase* in the loss function.

Intuition

Gradient descent iteratively updates parameters in order to minimize the loss function. Since the gradient points in the direction of steepest increase, we move in the *opposite* direction to descend the loss landscape:

$$\theta^{(k+1)} = \theta^{(k)} - (\text{step size}) \times (\text{gradient direction})$$

This method is foundational in optimization and learning algorithms, including training neural networks, fitting normalizing flows, and solving parameterized optimal transport problems.

Explanation of Density Matching in Continuous-Time Flow Models

This note describes a density-matching objective in the context of continuous-time optimal transport and flow-based generative modeling. We analyze the handwritten note line by line.

Line 1: Pushforward Map at Time t

$$T(\cdot, t)_\# \mu = F_t(\cdot)_\# \mu = \rho_t$$

This expression states that:

- There exists a continuous-time map $T(x, t)$, or equivalently $F_t(x)$, which pushes forward a base distribution μ to a new distribution ρ_t at time t .
- The notation $T_\# \mu$ denotes the *pushforward* of the measure μ under the map T , i.e., $\rho_t = T(\cdot, t)_\# \mu$.
- At each time t , the distribution ρ_t represents the current density resulting from transporting μ through the time-dependent map $T(\cdot, t)$.

Line 2: Goal — Match Target Distribution

$$\text{want } \rho_t \text{ close to target } \rho \text{ with samples } \{x_i\}_{i=1}^N \sim \rho$$

- The goal is to ensure that the time-evolving distribution ρ_t approaches a fixed target distribution ρ .
- It is assumed that we have access to data samples $x_1, \dots, x_N$ drawn i.i.d. from ρ .
- In the context of generative modeling, ρ typically represents the empirical data distribution, while ρ_t is the model's approximation at time t .

Line 3: Likelihood Objective

$$\Rightarrow \max \log \text{likelihood} \iff \min \text{negative log likelihood}$$

- The practical objective is to maximize the log-likelihood of the observed data samples under the model's distribution ρ_t :

$$\max \sum_{i=1}^N \log \rho_t(x_i)$$

- Equivalently, this is framed as minimizing the negative log-likelihood:

$$\min - \sum_{i=1}^N \log \rho_t(x_i)$$

- This formulation is standard in flow-based and diffusion-based models, where a base distribution μ is evolved over time to match a data distribution ρ .

At discrete step k , $T^{(k)} \# \mu = \rho^{(k)}$, want $\rho^{(k)} \rightarrow \rho$
 $\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla L(\theta) |_{\theta^{(k)}}$, $T^{(k)} = T(\cdot; \theta^{(k)})$

$$L(\theta^{(k)}) = \sum_{i=1}^N -\log(\mu(T^{-1}(x_i; \theta^{(k)}))) - \log |J_{T^{-1}}(T^{-1}(x_i; \theta^{(k)}))|$$

 given $\mu \sim \text{std Gauss}$, $\mu(x) = C \cdot e^{-\frac{\|x\|^2}{2}}$
 $\Rightarrow L(\theta^{(k)}) = \sum_{i=1}^N \frac{\|T^{-1}(x_i; \theta^{(k)})\|^2}{2} - \log |J_{T^{-1}}(T^{-1}(x_i; \theta^{(k)}))|$
 $\hookrightarrow \text{const. doesn't matter}$

Derivation of the Negative Log-Likelihood Loss in Normalizing Flows

We derive the expression for the negative log-likelihood loss used in flow-based generative models. These models aim to learn an invertible transformation $T(\cdot; \theta)$ that pushes forward a simple base distribution μ (typically a standard Gaussian) to match a target data distribution ρ .

Change-of-Variables Formula

Let $\{x_i\}_{i=1}^N \sim \rho$ be observed data samples. We define a parameterized invertible map $T(\cdot; \theta)$ such that the pushforward of μ under T approximates ρ :

$$T_{\#} \mu \approx \rho.$$

By the change-of-variables formula, the model density $\rho_{\theta}(x)$ is given by:

$$\rho_{\theta}(x) = \mu(T^{-1}(x; \theta)) \cdot |\det J_{T^{-1}}(x; \theta)|,$$

where $J_{T^{-1}}(x; \theta)$ denotes the Jacobian of the inverse transformation evaluated at x .

Log-Likelihood of the Data

The log-likelihood of the data under the model is:

$$\sum_{i=1}^N \log \rho_{\theta}(x_i) = \sum_{i=1}^N \log \mu(T^{-1}(x_i; \theta)) + \log |\det J_{T^{-1}}(x_i; \theta)|.$$

Negative Log-Likelihood Loss

In practice, we minimize the negative log-likelihood:

$$L(\theta) := - \sum_{i=1}^N \log \rho_{\theta}(x_i) = \sum_{i=1}^N -\log \mu(T^{-1}(x_i; \theta)) - \log |\det J_{T^{-1}}(x_i; \theta)|.$$

This loss function corresponds to the first boxed equation in the handwritten note.

Specialization to Standard Gaussian Base

Assume the base distribution μ is a standard Gaussian:

$$\mu(z) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|z\|^2}{2}\right).$$

Taking the log and ignoring the additive constant $-\frac{d}{2} \log(2\pi)$, which does not depend on θ , we obtain:

$$-\log \mu(z) = \frac{\|z\|^2}{2} + (\text{const}).$$

Here, z represents the latent variable in the base space, obtained by pulling back the data point x through the inverse transformation:

$$z = T^{-1}(x; \theta).$$

Importantly, z is not the same as θ ; θ denotes the parameters of the transformation T , which are optimized during training. The variable $z \in \mathbb{R}^d$ depends on both x and θ , but it lives in the latent space where μ is defined.

Substituting into the loss:

$$L(\theta) = \sum_{i=1}^N \frac{\|T^{-1}(x_i; \theta)\|^2}{2} - \log |\det J_{T^{-1}}(T^{-1}(x_i; \theta))|.$$

This matches the second boxed equation in the handwritten note.

- Deep Learning (Neural Network) x

- Fully-Connected DNNs:

- Depth: L

- Width: $w_0, w_1, \dots, w_L$

- Activation:

$$q_e(x; \theta) \in \mathbb{R}^{w_e}$$

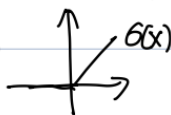
$\hookrightarrow$ layer e

Input: $q_0(x; \theta) = x$

$$q_e(x; \theta) = \sigma(W_e q_{e-1}(x; \theta) + b_e)$$

- $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ (applied element-wise)

is nonlinear. Eg. ReLU, sigmoid

$$\sigma(x) = \max(x, 0)$$


- Weight matrices: $W_e \in \mathbb{R}^{w_e \times w_{e-1}}$

- bias vector: $b_e \in \mathbb{R}^{w_e}$

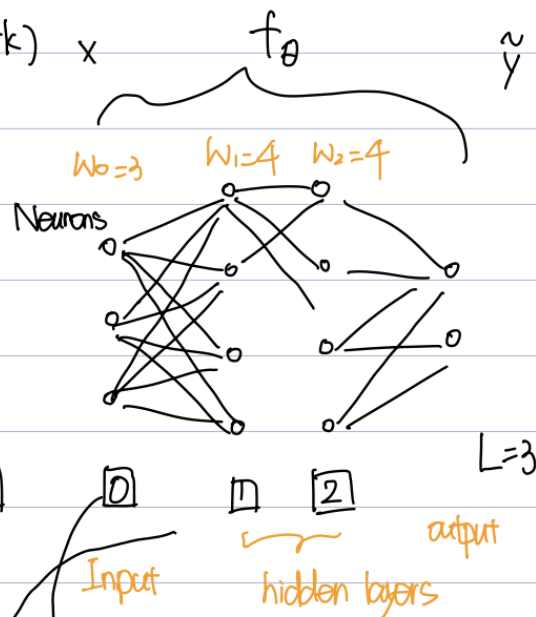
- $\theta = (W_1, b_1, \dots, W_L, b_L) \Leftarrow$ Gradient Descent on θ

$$f_\theta(x) = f(x; \theta) = \tilde{y}$$

purely a notational difference.

$$\text{Loss: } L(y, \tilde{y}) \Rightarrow \min L(y, \tilde{y})$$

$$= \min_{\theta} L(y, f(x; \theta))$$



Input Neurons: activation of input

Neurons: activation w/ weighted sum of neurons from previous layer

$\rightarrow$ Nonlinear func.

Input neurons simply pass values forward without applying any activation function. In contrast, all other neurons in the hidden and output layers first compute a weighted sum of inputs from the previous layer and then apply a nonlinear activation function, such as ReLU or sigmoid, to produce their output. This nonlinearity is essential, as it gives neural networks their expressive power and enables them to approximate complex, nonlinear functions.

Layerwise Activation Vector: $q_\ell(x; \theta) \in \mathbb{R}^{w_\ell}$

Notation Breakdown

- $q_\ell(x; \theta) \in \mathbb{R}^{w_\ell}$ denotes the output vector (activation) of the ℓ -th layer in a fully connected deep neural network.
- $x \in \mathbb{R}^{w_0}$ is the input vector to the network.
- θ is the collection of all learnable parameters: weight matrices and bias vectors for each layer.
- w_ℓ is the *width* of layer ℓ , i.e., the number of neurons in that layer.
- The activation is computed recursively using:

$$q_\ell(x; \theta) = \sigma(W_\ell q_{\ell-1}(x; \theta) + b_\ell), \quad \text{for } \ell = 1, \dots, L$$

where $W_\ell \in \mathbb{R}^{w_\ell \times w_{\ell-1}}$ is the weight matrix, $b_\ell \in \mathbb{R}^{w_\ell}$ is the bias vector, and σ is a nonlinear activation function (e.g., ReLU, sigmoid).

Concrete Example

Consider a simple feedforward network with the following structure:

- Input layer: $w_0 = 2$ neurons.
- Hidden layer ($\ell = 1$): $w_1 = 3$ neurons.
- Activation function: ReLU $\sigma(z) = \max(0, z)$ applied elementwise.

Let the input be:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \text{so } q_0(x; \theta) = x \in \mathbb{R}^2.$$

Suppose the parameters of layer 1 are:

$$W_1 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -3 & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad b_1 = \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} \in \mathbb{R}^3.$$

Then the pre-activation vector is:

$$W_1 q_0(x; \theta) + b_1 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1(1) + (-1)(2) \\ 0(1) + 2(2) \\ -3(1) + 1(2) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 4 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 5 \\ -2 \end{bmatrix}$$

Now apply the ReLU activation:

$$q_1(x; \theta) = \sigma(W_1 q_0(x; \theta) + b_1) = \max(0, \cdot) = \begin{bmatrix} 0 \\ 5 \\ 0 \end{bmatrix} \in \mathbb{R}^3$$

Thus, the activation vector $q_1(x; \theta) \in \mathbb{R}^3$ represents the output of the first hidden layer after applying weights, bias, and nonlinearity.

✓ $f_{\theta}(x)$ — Entire Network Function

- Represents the **complete neural network function**, from input to output.
- It's defined as:

$$f_{\theta}(x) = q_L(x; \theta)$$

- It uses all intermediate q_{ℓ} 's to build the final prediction $\hat{y}$.

↻ Summary of the Relationship

Symbol	Meaning
$q_{\ell}(x; \theta)$	Output of layer ℓ (intermediate activation)
$f_{\theta}(x)$	Full network output = $q_L(x; \theta)$

$\{(x_i, y_i)\}_{i=1}^N$ try to find f st. $f(x) = y$

Training data